PRAKTIK PEMROGRAMAN · 90 MENIT

Bangun vending simulasi dari nol

Python dasar, tombol Tkinter, keranjang, lalu pengantar serial Arduino

01 · DATA
Produk

02 · ATURAN
Keranjang

03 · TAMPILAN
Tkinter

04 · PERANGKAT
Serial

Mulai dengan aplikasi latihan; jangan memakai API key atau motor.

Pada akhir praktik, mahasiswa bisa…

Tidak harus hafal semua kode; pahami hubungan antara data, fungsi, tombol, dan hasil.

MENULIS

Membuat daftar produk, aturan stok/total, dan jendela sederhana dari file kosong.

MENGUJI

Menjalankan skrip, membaca pesan error, dan membuktikan perubahan.

MENJELASKAN

Mengikuti alur tombol → aturan keranjang → status aplikasi.

MENGENAL PERANGKAT

Memahami pesan teks Python–Arduino tanpa harus menyalakan motor.

Dari folder kosong ke vending simulasi

Kita ketik program kecil bersama, lalu lihat cara proyek mengendalikan perangkat.

00–08	Tujuan, folder latihan, dan file Python
08–20	Menulis data produk di inventory.py
20–32	Menulis aturan keranjang dan mengujinya
32–55	Membuat UI tombol di main.py
55–65	Menjalankan aplikasi dan tantangan produk
65–90	Protokol serial, sketch, dan demo aman

Target: simulasi dari nol; QRIS dan motor nyata bukan latihan mandiri.

Satu pilihan produk melewati beberapa tahap

Program mini yang kita bangun berhenti pada checkout simulasi.

01 · PILIH

Tombol produk

02 · CEK

Stok tersedia

03 · HITUNG

Harga keranjang

04 · SIMULASI

Status pembayaran

05 · OPSIONAL

Serial ke board

BATAS LATIHAN

Checkout latihan hanya menampilkan status SIMULASI. Tidak membuat transaksi QRIS sungguhan.

Buat folder latihan yang terpisah

Jangan mengubah .env atau menimpa file aplikasi utama saat latihan coding dari nol.

DI VS CODE

Pilih File → Open Folder… lalu buat/buka folder bernama latihan_vending.

BUAT FILE

inventory.py, cart.py, test_cart.py, dan main.py. Perhatikan ejaan dan ekstensi.

```
py --version
```

CEK TERMINAL

Buka Terminal → New Terminal di VS Code. Jika perintah di atas menampilkan versi Python, lanjut.

Siapkan folder dan buat file pertama

Lakukan kliknya berurutan; semua latihan ditulis di folder baru, bukan di proyek vending utama.

LANGKAH DI VS CODE

1. File → Open Folder…
2. Buat/pilih folder latihan_vending
3. Explorer → New File
4. Buat inventory.py

BUAT FILE SATU PER SATU

Untuk sekarang cukup inventory.py. File lain dibuat saat langkahnya tiba.

```
py --version
```

CEK

Terminal → New Terminal. Jalankan perintah di atas. Jika versi Python muncul, lanjut.

Buat variabel PRODUCTS yang masih kosong

File: inventory.py · Ganti seluruh isi file dengan tepat satu baris berikut.

```
PRODUCTS = []
```

KETIK DAN SIMPAN

Klik inventory.py → Ctrl+A → ketik baris di atas → Ctrl+S.

JALANKAN

Terminal: py -c "from inventory import PRODUCTS; print(PRODUCTS)"

HASIL YANG HARUS MUNCUL: []

Masukkan produk Air Mineral

File: inventory.py · Ganti seluruh baris PRODUCTS = [] dengan kode ini.

```
PRODUCTS = [
    {"name": "Air Mineral", "price": 3000, "stock": 5, "slot": 1},
]
```

SIMPAN DAN UJI

Ctrl+S. Terminal: py -c "from inventory import PRODUCTS; print(PRODUCTS)"

HASIL YANG DIHARAPKAN

Daftar berisi Air Mineral, harga 3000, stok 5, slot 1.

name = nama barang · price = harga · stock = stok · slot = jalur dispenser

Tambahkan dua produk lagi

File: inventory.py · Ganti seluruh isi file; perhatikan koma setelah setiap produk.

```
PRODUCTS = [
    {"name": "Air Mineral", "price": 3000, "stock": 5, "slot": 1},
    {"name": "Kopi", "price": 5000, "stock": 3, "slot": 2},
    {"name": "Teh Botol", "price": 4000, "stock": 4, "slot": 3},
]
```

JALANKAN

Tekan Ctrl+S. Terminal: py -c "from inventory import PRODUCTS; print(len(PRODUCTS))"

CEK HASIL

Terminal harus menampilkan angka 3. Slot harus berbeda: 1, 2, 3.

Buat cart.py dan fungsi tambah produk

Explorer → New File → ketik cart.py · Masukkan kode berikut, lalu Ctrl+S.

```
def add_to_cart(products, cart, index):
    if products[index]["stock"] <= 0:
        return False
    products[index]["stock"] -= 1
    cart.append(index)
    return True
```

ARTI BARISNYA

Cek stok. Jika habis, kembalikan False. Jika ada, kurangi stok dan catat index produk.

CEK SINTAKS

Terminal: py -m py_compile cart.py
Tidak ada output berarti berhasil.

Tambahkan fungsi untuk menghitung total

File: cart.py · Letakkan kursor di akhir file. Tambahkan fungsi di bawah add_to_cart.

```
def calculate_total(products, cart):
    return sum(products[index]["price"] for index in cart)
```

SIMPAN

Ctrl+S. Jangan menghapus fungsi add_to_cart yang sudah dibuat.

CEK

Jalankan py -m py_compile cart.py lagi. Tidak ada output = sintaks benar.

sum(...) menjumlahkan harga produk yang index-nya tersimpan di cart.

Tambahkan fungsi pembatalan pesanan

File: cart.py · Tambahkan fungsi ini paling bawah; jangan mengganti fungsi sebelumnya.

```
def cancel_order(products, cart):
    for index in cart:
        products[index]["stock"] += 1
    cart.clear()
```

APA YANG TERJADI?

Untuk setiap index di pesanan, stok dikembalikan satu. Lalu cart dikosongkan.

CEK FILE

Ctrl+S → py -m py_compile cart.py. Pastikan tidak ada error.

Buat test_cart.py dan jalankan tes

Explorer → New File → ketik test_cart.py · Masukkan seluruh kode, simpan, lalu jalankan.

```
from inventory import PRODUCTS
from cart import add_to_cart, calculate_total, cancel_order

cart = []
assert add_to_cart(PRODUCTS, cart, 0)
assert PRODUCTS[0]["stock"] == 4
assert calculate_total(PRODUCTS, cart) == 3000
cancel_order(PRODUCTS, cart)
assert PRODUCTS[0]["stock"] == 5
assert cart == []
print("Semua uji keranjang lulus")
```

JALANKAN

Terminal: py test_cart.py

HASIL

Semua uji keranjang lulus

Buat main.py dan jendela kosong

Explorer → New File → ketik main.py · Masukkan kode ini; ini checkpoint pertama untuk tampilan.

```
import tkinter as tk

root = tk.Tk()
root.title("Latihan Vending")
root.geometry("440x480")

tk.Label(root, text="Mini Vending").pack(pady=20)

root.mainloop()
```

JALANKAN

Terminal: py main.py. Tutup jendela dengan tombol X.

HASIL

Jendela 440×480 muncul dengan judul Mini Vending.

Tambahkan data dan label total

File: main.py · Tambahkan kode pada posisi yang disebut; jangan hapus root.mainloop().

```
# Tambahkan di bawah import tkinter:
from inventory import PRODUCTS
from cart import add_to_cart, calculate_total, cancel_order

# Tambahkan sebelum root = tk.Tk():
cart = []

# Tambahkan setelah tk.Label judul:
total = tk.StringVar(value="Total: Rp 0")
status = tk.StringVar(value="Pilih produk")
products_frame = tk.Frame(root)
products_frame.pack(fill="x", padx=20)
tk.Label(root, textvariable=total).pack(pady=10)
tk.Label(root, textvariable=status).pack(pady=8)
```

SIMPAN DAN JALANKAN

Gambarkan satu tombol untuk setiap produk

File: main.py · Tambahkan fungsi ini tepat sebelum root.mainloop().

```
def refresh_products():
    for widget in products_frame.winfo_children():
        widget.destroy()
    for index, product in enumerate(PRODUCTS):
        text = f'{product["name"]} - Rp {product["price"]}'
        button = tk.Button(
            products_frame,
            text=text,
            command=lambda: status.set("Tombol berhasil diklik"),
        )
        button.pack(fill="x", pady=4)

refresh_products()
```

POSISI KODE

Fungsi dan pemanggilannya harus di atas root.mainloop().

UJI

Simpan, jalankan. Tiga tombol produk muncul; klik dan cek pesan.

Hubungkan klik ke fungsi add_to_cart

File: main.py · Sekarang ganti perintah tombol dan tambahkan fungsi pemilih.

```
# Di dalam tk.Button, ganti baris command= menjadi:
command=lambda i=index: select_product(i),

# Tambahkan sebelum refresh_products():
def select_product(index):
    added = add_to_cart(PRODUCTS, cart, index)
    if added:
        status.set(f'{PRODUCTS[index]["name"]} ditambahkan')
    else:
        status.set("Stok habis")
    amount = calculate_total(PRODUCTS, cart)
    total.set(f"Total: Rp {amount:,}".replace(",", "."))
    refresh_products()
```

UJI

Simpan, jalankan, klik Air Mineral. Total menjadi Rp 3.000 dan stok berkurang.

Tampilkan stok terbaru dan tombol batal

File: main.py · Perbarui teks tombol setiap kali refresh_products dijalankan.

```
# Ganti teks tombol di refresh_products() dengan:
text = (
    f'{product["name"]} - Rp {product["price"]} '
    f'(stok: {product["stock"]})'
)

# Tambahkan sebelum refresh_products():
def cancel_cart():
    cancel_order(PRODUCTS, cart)
    total.set("Total: Rp 0")
    status.set("Pesanan dibatalkan; stok kembali")
    refresh_products()

# Tambahkan setelah tk.Label status:
tk.Button(root, text="Batalkan pesanan",
          command=cancel_cart).pack(pady=5)
```

UJI

Tambahkan tombol bayar simulasi

File: main.py · Tidak ada API, QRIS, atau uang sungguhan dalam latihan ini.

```
# Tambahkan sebelum refresh_products():
def simulate_payment():
    if not cart:
        status.set("Keranjang masih kosong")
        return
    amount = calculate_total(PRODUCTS, cart)
    cart.clear()
    total.set("Total: Rp 0")
    status.set(f"SIMULASI: pembayaran Rp {amount:,} diterima")
    refresh_products()

# Tambahkan setelah tombol Batalkan pesanan:
tk.Button(root, text="Bayar (simulasi)",
          command=simulate_payment).pack(pady=5)
```

JALANKAN

Simpan → tutup jendela → py main.py

UJI

Pilih produk lalu bayar. Pesan SIMULASI muncul; stok terjual tidak dipulihkan.

Periksa stok habis dan kembalikan stok

File: test_cart.py · Tambahkan tes ini paling bawah untuk memastikan batas stok aman.

```
limited = [{"name": "Permen", "price": 1000, "stock": 1}]
order = []
assert add_to_cart(limited, order, 0) is True
assert add_to_cart(limited, order, 0) is False
assert calculate_total(limited, order) == 1000
cancel_order(limited, order)
assert limited[0]["stock"] == 1
assert order == []
print("Tes stok habis lulus")
```

JALANKAN

Terminal: py test_cart.py

HASIL

Dua pesan lulus: uji keranjang dan tes stok habis.

Tambahkan satu produk dan demonstrasikan

File: inventory.py · Tambahkan satu baris produk sebelum tanda] terakhir.

```
    {"name": "Cokelat", "price": 6000, "stock": 2, "slot": 4},
```

URUTAN AKHIR

Ctrl+S → py test_cart.py → py main.py

HASIL

Tombol Cokelat muncul otomatis tanpa menulis tombol baru.

SETELAH BERHASIL

Baru sekarang bandingkan inventory.py/cart.py/main.py latihan dengan modul bernama sama di proyek vending utama.

Program latihan kecil, aplikasi utama modular

Konsepnya sama; proyek utama memisahkan tanggung jawab lebih rinci agar mudah dirawat.

Latihan dari nol	→	Proyek vending utama
inventory.py	→	inventory.py · data dan stok
cart.py	→	cart.py · aturan pesanan
main.py berisi UI mini	→	main.py + vending_ui.py · alur dan tampilan
Pembayaran simulasi	→	QRIS/API · konfigurasi dan checkout
Belum mengendalikan motor	→	serial_controller.py + firmware Arduino

JANGAN CAMPUR FOLDER · Ketik latihan di latihan_vending; gunakan proyek utama untuk mempelajari integrasinya.

Klik produk sampai perintah slot

Cari fungsi satu demi satu; tidak perlu membaca seluruh main.py sekaligus.

```
item = add_to_cart(products, order_data["items"], index)
if item is None:
    return

order_data["total_amount"] += item["price"]
update_products()
update_order()
save_stock()

# sesudah pembayaran berhasil:
serial_controller.dispense(item["slot"])
```

TELUSURI

Ctrl+Shift+F → cari select_product, lalu add_to_cart.

INGAT

Pemilihan produk belum berarti pembayaran berhasil atau motor

Latihan QRIS sandbox tanpa Arduino

MODE_DEMO melewati dispenser; SUMOPOD_SANDBOX menjaga pembayaran tetap di lingkungan uji.

```
# Buka .env proyek vending, tambahkan:
MODE_DEMO=1
SUMOPOD_SANDBOX=1

# Simpan, tutup aplikasi, lalu jalankan ulang:
py main.py
```

PASTIKAN MODE AKTIF

Status menampilkan MODE DEMO · SumoPod Sandbox. Arduino tidak diperlukan.

COBA BAYAR

Dengan API key sandbox, pindai QR sandbox. Tanpa key, aplikasi memakai simulasi lokal.

AMAN

Demo tidak pernah memakai endpoint production. Pastikan API key berasal dari sandbox; pembayaran sandbox tidak mengeluarkan barang.

Sepakati format pesan sebelum menulis kode

Python dan Arduino bertukar teks per baris melalui port COM.

BOARD → PYTHON	`READY`	Kontroler selesai mulai.
PYTHON → BOARD	`PING`	Minta pemeriksaan respons.
BOARD → PYTHON	`PONG`	Kontroler merespons.
PYTHON → BOARD	`DISPENSE,2`	Minta produk dari slot 2.
BOARD → PYTHON	`DONE,2 / ERROR`	Selesai atau perintah ditolak.

Setiap pesan diakhiri newline; kedua sisi memakai baud rate yang sama.

Kirim permintaan dan periksa jawaban

Contoh konsep singkat; proyek utama memakai timeout dan penanganan error yang lebih lengkap.

```
import serial

def dispense(port, slot):
    if slot < 1 or slot > 6:
        raise ValueError("Slot harus 1 sampai 6")

    with serial.Serial(port, 9600, timeout=2) as board:
        ready = board.readline().decode("utf-8").strip()
        if ready != "READY":
            raise RuntimeError(f"Kontroler belum siap: {ready}")
        board.write(f"DISPENSE,{slot}\n".encode("ascii"))
        response = board.readline().decode("utf-8").strip()
        expected = f"DONE,{slot}"
        if response != expected:
            raise RuntimeError(f"Respons tidak sesuai: {response}")
```

KONTRAK

Slot pada jawaban harus sama dengan slot yang diminta.

PORT COM

Pilih COM yang terdeteksi; jangan menebak.

Tetapkan pin dan kondisi aman saat mulai

Contoh pin untuk Uno; bandingkan dengan pemetaan board di firmware proyek.

```
const uint8_t SLOT_COUNT = 6;
const unsigned long DISPENSE_DURATION_MS = 1000;
const bool RELAY_ACTIVE_LOW = true;
const uint8_t RELAY_ON = RELAY_ACTIVE_LOW ? LOW : HIGH;
const uint8_t RELAY_OFF = RELAY_ACTIVE_LOW ? HIGH : LOW;
const uint8_t RELAY_PINS[SLOT_COUNT] = {2, 3, 4, 5, 6, 7};

void setup() {
  Serial.begin(9600);
  for (uint8_t i = 0; i < SLOT_COUNT; i++) {
    digitalWrite(RELAY_PINS[i], RELAY_OFF);
    pinMode(RELAY_PINS[i], OUTPUT);
  }
  Serial.println("READY");
}
```

WAJIB COCOKKAN

Pin, jenis board, relay aktif-LOW/HIGH, tegangan, dan rangkaian nyata harus diverifikasi sebelum memberi daya.

Baca dan validasi perintah sebelum akses pin

Jangan gunakan nilai slot dari teks sebelum memastikan rentangnya valid.

```
String command = Serial.readStringUntil('\n');
command.trim();

if (command == "PING") {
  Serial.println("PONG");
  return;
}
if (!command.startsWith("DISPENSE,")) {
  Serial.println("ERROR,UNKNOWN_COMMAND");
  return;
}
int slot = command.substring(9).toInt();
if (slot < 1 || slot > SLOT_COUNT) {
  Serial.println("ERROR,INVALID_SLOT");
  return;
}
```

PENCEGAHAN

Validasi slot mencegah indeks di luar daftar pin.

Aktifkan relay sebentar lalu kirim DONE

Pesan DONE hanya mengonfirmasi program menyelesaikan durasi output.

```
uint8_t relayPin = RELAY_PINS[slot - 1];
digitalWrite(relayPin, RELAY_ON);
delay(DISPENSE_DURATION_MS);
digitalWrite(relayPin, RELAY_OFF);

Serial.print("DONE,");
Serial.println(slot);
```

PERHATIKAN INDEKS

Slot dimulai dari 1; indeks array dimulai dari 0.

BATAS KONFIRMASI

DONE bukan sensor yang memastikan barang benar-benar jatuh.

Uji dulu tanpa board dan motor

FakeSerial menirukan koneksi untuk memastikan bentuk pesan dan respons.

```
py test_cart.py
py test_serial_controller.py
py test_main_gui.py
```

UJI TIRUAN

Dapat memeriksa READY, PING/PONG, format DISPENSE, dan DONE.

BUKAN UJI HARDWARE

Tidak membuktikan kabel, COM, tegangan, relay, pin, atau motor bekerja.

Perangkat fisik hanya dengan pengawasan instruktur

Mahasiswa tidak perlu melakukan pembayaran live atau mengaktifkan motor sendiri.

01 Periksa board, pin, relay, sumber daya, dan mekanik.

02 Hubungkan board; temukan COM yang benar di Device Manager.

03 Tutup Serial Monitor sebelum program Python memakai port yang sama.

04 Uji tanpa produk berbahaya dan jauhkan tangan dari mekanisme.

05 Hentikan bila relay aktif saat startup atau wiring tidak cocok.

TANPA TRANSAKSI LIVE

Mulai dari simulasi; jangan memproyeksikan atau membagikan isi .env.

Ceritakan kembali alurnya dengan kata sendiri

Satu perubahan → jalankan tes → amati → baru lanjut.

TANTANGAN

Tambahkan produk di inventory.py latihan. Uji tombol, harga, dan stok di main.py.

EXIT TICKET

File mana menyimpan data? Fungsi mana memeriksa stok? Apa arti DONE,2?

TROUBLESHOOTING

ModuleNotFoundError: cek folder kerja dan nama file. SyntaxError: cek tanda baca/indentasi. Timeout serial: cek COM, baud rate, dan koneksi.

Terima kasih · gunakan simulasi untuk praktik mandiri.